

GamutX

Functions

Python Basics



Introduction to Functions

- What is a Function?
- Creating a Function
- Calling a Function
- What are Parameters and Arguments?

What is a Function?

- A function is a reusable block of code that performs a specific task.
- Functions allow you to organize your code into logical sections, making it easier to read, maintain, and reuse.

🎯 Why Use Functions?

Reusability

- We can write code once and use it multiple times.

Modularity

- We can break your big program into smaller, manageable parts.

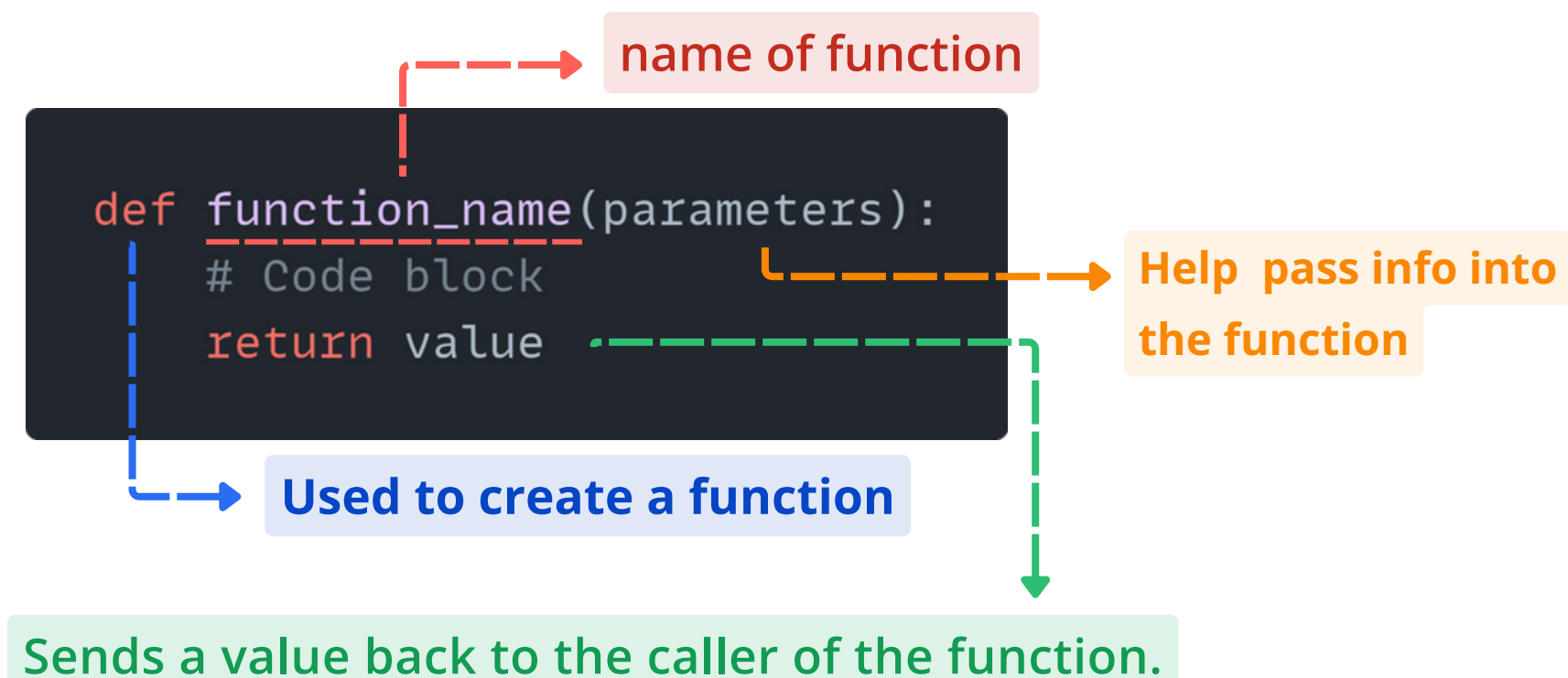
Abstraction

- We can hide the complex logic and just use a simple function name.

Creating a Function

- In Python, we use the **def keyword to define a function**.

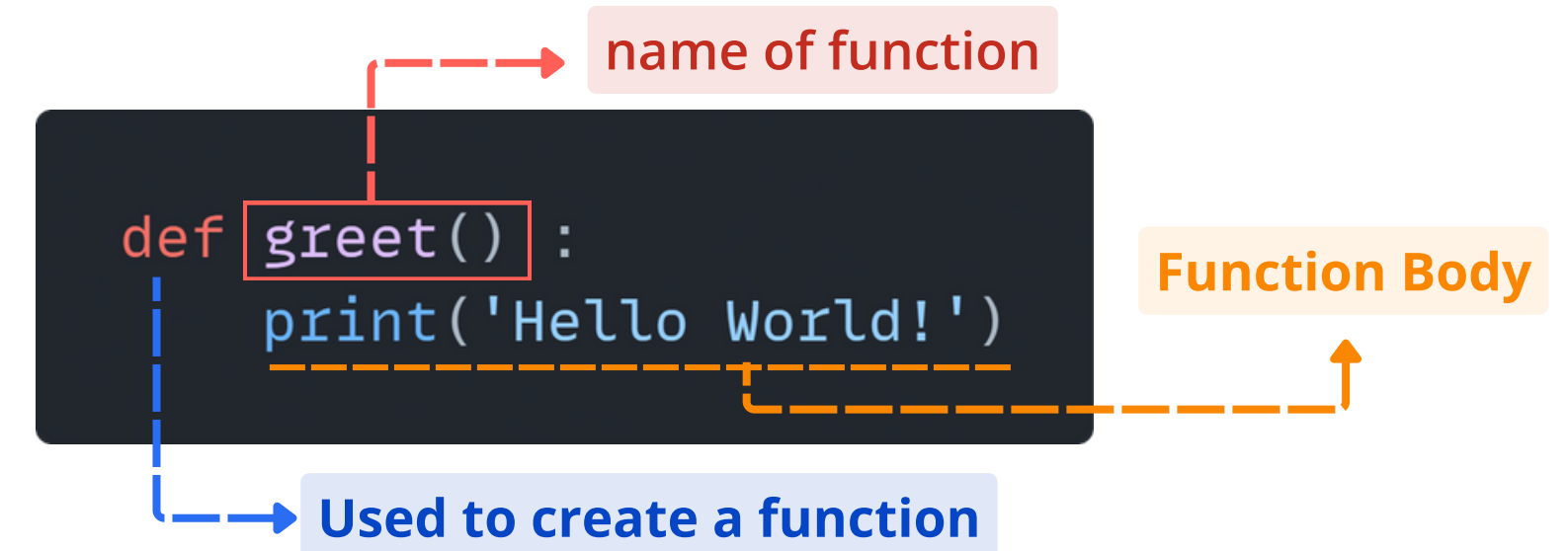
Basic Syntax



Rules for naming Function:

- Starts with a **letter or underscore**
- Can **include letters, numbers, underscores**
- Should be descriptive** (e.g., `calculate_sum`)

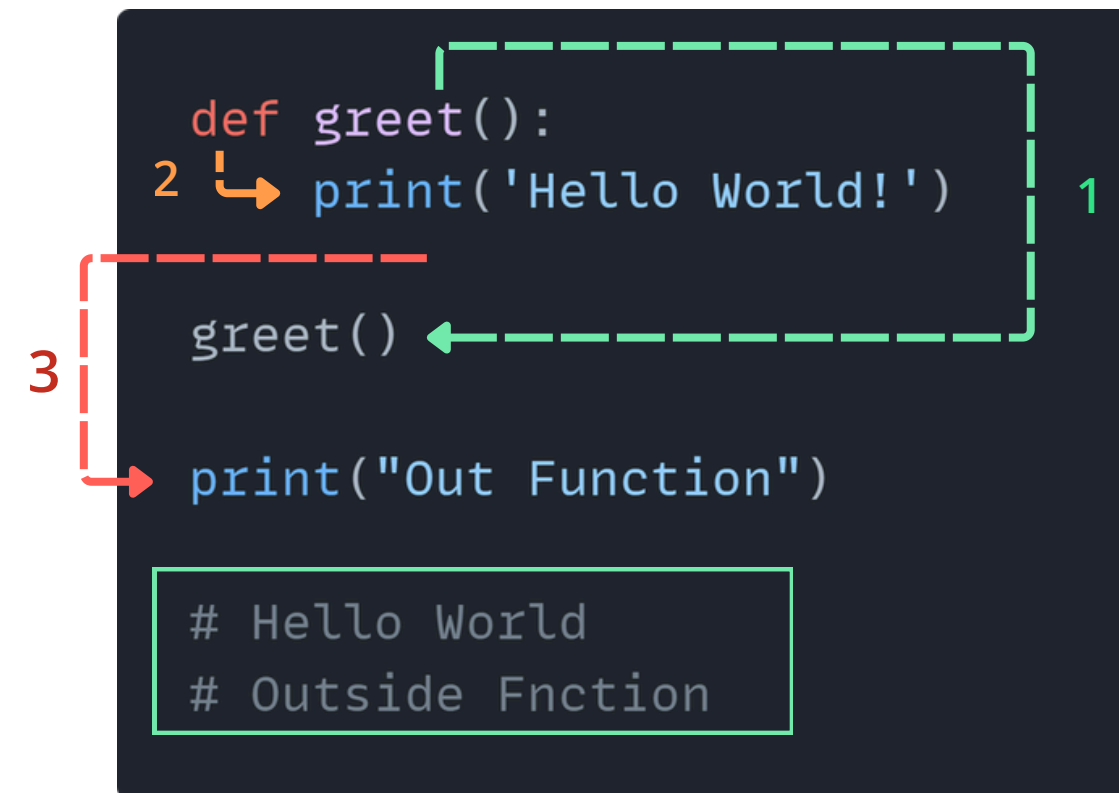
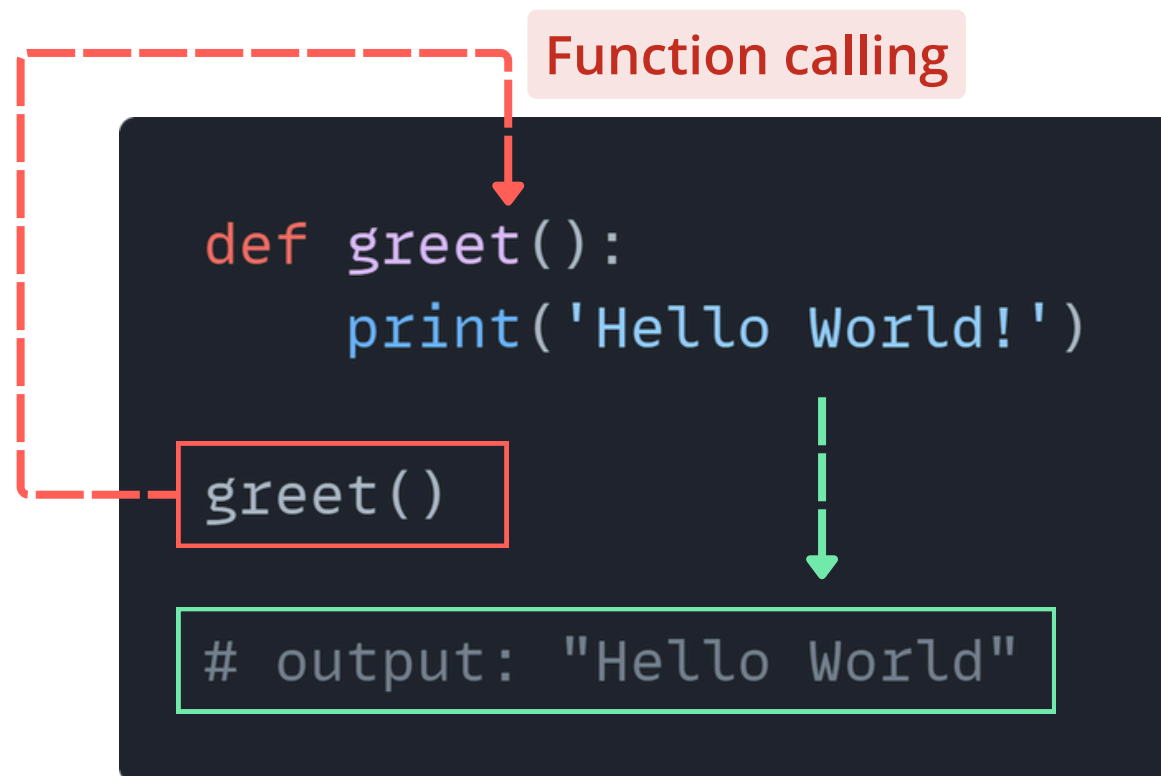
Creating a Function



Calling a Function

- Defining a function doesn't run it — we need to call it.

Basic Usage



How Python Executes This Code:

- Python **first** sees `greet()` and calls the function.
- It **then** jumps to the `greet()` function and runs the code inside it.
- After the **function** finishes, python runs `print("Outside Function")`.

Concept Practice Task

Function to Print a Simple Goodbye Message

```
def say_goodbye():  
    print("Goodbye! Have a great day!")  
  
say_goodbye()
```

```
# Output:  
# Goodbye! Have a great day!
```

Function for Simple Addition

```
def add_numbers():  
    a = 5  
    b = 3  
    print("The sum is:", a + b)
```

```
add_numbers()  
# The sum is: 8
```

Function to Show a Count Down

```
def countdown():  
    count = 5  
    while count > 0:  
        print(count)  
        count -= 1  
  
countdown()
```

```
# Output:  
# 5  
# 4  
# 3  
# 2  
# 1
```

Function to Print Even Numbers from 1 to 10

```
def print_even_numbers():  
    number = 2  
    while number <= 10:  
        print(number)  
        number += 2  
  
print_even_numbers()
```

```
# Output:  
# 2  
# 4  
# 6  
# 8  
# 10
```

What are Parameters & Arguments

- When you use functions, you often want to send some data into the function.
- That's where parameters and arguments come in.

Basic Usage

```
def greet(name):  
    print("Hello", name)  
  
greet("John")  
  
# Hello John
```

Parameters

- A **placeholder name** used in a function to **receive input**.
 - 💡 Here, name is the parameter
 - no value yet, just a **placeholder**.

Arguments

- The **real value** we give to the function **when we call it**.
- It **replaces the parameter during execution**.
 - 💡 Here, John is the argument
 - it gets passed to name, and the function prints it.

Function call

Function definition

Concept Practice Task

GamutX

Function with One Parameter and Argument

```
def greet(name):  
    print(f"Hello, {name}!")  
  
greet("Alice")
```

```
# Output:  
# Hello, Alice!
```

Function with Two Parameters and Arguments

```
def add(a, b):  
    print(f"The sum of {a} and {b} is:", a + b)  
  
add(3, 4)
```

```
# Output:  
# The sum of 3 and 4 is: 7
```

Function with Three Parameters and Arguments

```
def introduce(name, age, city):  
    print(f"My name is {name}, I'm {age} years old, and I live in {city}.")  
  
introduce("John", 25, "London")
```

```
# Output:  
# My name is John, I'm 25 years old, and I live in London.
```

Function Arguments

- Types of Function Arguments
- Positional Arguments
- Keyword Arguments
- Default Arguments

Types of Function Arguments

- When calling a function, we can pass arguments in different ways

Positional Arguments

- Values are passed in the **same order** as the **parameters**.

Keyword Arguments

- Each value is **assigned** using the **parameter name**.

Default Arguments

- Parameters that have **default values** if **no argument** is provided.

Arbitrary Arguments

- ***args** — for multiple non-keyword arguments
- ****kwargs** — for multiple keyword arguments

Note: We'll cover ***args** and ****kwargs** later in the intermediate level.

Positional Arguments

- These are the **simplest type of arguments**.
- In this **values are passed in the same order as the parameters**.

Basic Usage

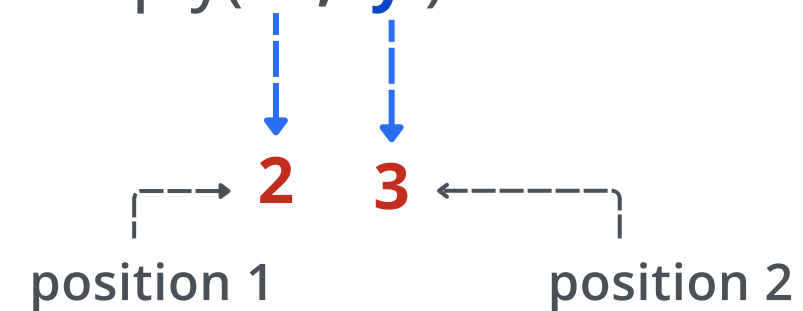
```
def multiply(x, y):  
    print(x * y)
```

```
multiply(2, 3)
```

```
# Output: 6
```

Here

```
def multiply(x, y):
```



Concept Practice Task

GamutX

Introducing a Person

```
def introduce_person(name, age):  
    print(f"My name is {name} and I am {age} years old.")  
  
introduce_person("Alice", 30)
```

```
# Output:  
# My name is Alice and I am 30 years old.
```

Displaying Book Information

```
def book_info(title, author, year):  
    print(f"'{title}' by {author}, published in {year}.")  
  
book_info("1984", "George Orwell", 1949)
```

```
# Output:  
# '1984' by George Orwell, published in 1949.
```

Calculating the Perimeter of a Rectangle

```
def calculate_perimeter(length, width):  
    perimeter = 2 * (length + width)  
    print(f"The perimeter of the rectangle is {perimeter} units.")  
  
calculate_perimeter(10, 5)
```

```
# Output:  
# The perimeter of the rectangle is 30 units.
```

Multiplying Numbers

```
def multiply(a, b, c):  
    print(f"The result of multiplying {a}, {b}, and {c} is:", a * b * c)  
  
multiply(2, 3, 4)
```

```
# Output:  
# The result of multiplying 2, 3, and 4 is: 24
```


Concept Practice Task

Food Order Example

```
def food_order(main_course, side_dish, drink):  
    print(f"You ordered {main_course} with {side_dish} and a {drink}.")  
  
food_order("Burger", "Fries", "Coke")
```

```
# Output:  
# You ordered Burger with Fries and a Coke.
```

Showing Purchase Details

```
def purchase(item, quantity, price):  
    print(f"You bought {quantity} {item}(s) at ${price} each.")  
  
purchase("notebook", 3, 2.5)
```

```
# Output:  
# You bought 3 notebook(s) at $2.5 each.
```

Keyword Arguments

- We **specify the name of each parameter** when calling the function.
- So we **don't have to follow the order** used in the function definition

Basic Usage

```
def greet(name, age):  
    print(f"Hello, {name}. You are {age} years old.")  
  
greet(age=25, name="Alice")  
  
# Output: Hello, Alice. You are 25 years old.
```

- **age=25** → goes to age
- **name="Alice"** → goes to name

Why Use Keyword Arguments?

- Makes the code more clear and readable
- Lets you change the order of arguments

Concept Practice Task

Introducing a Person

```
def introduce_person(name, age):  
    print(f"My name is {name} and I am {age} years old.")  
  
introduce_person(name="Alice", age=30)
```

```
# Output:  
# My name is Alice and I am 30 years old.
```

Displaying Book Information

```
def book_info(title, author, year):  
    print(f"'{title}' by {author}, published in {year}.")  
  
book_info(title="1984", author="George Orwell", year=1949)
```

```
# Output:  
# '1984' by George Orwell, published in 1949.
```

Displaying Personal Details

```
def person_details(name, age, city):  
    print(f"{name} is {age} years old and lives in {city}.")  
  
person_details(name="John", age=25, city="New York")
```

```
# Output:  
# John is 25 years old and lives in New York.
```

Concept Practice Task

Sending an Email Notification

```
def send_email(subject, recipient, message):  
    print(f"Sending email to {recipient} with subject '{subject}' and message: {message}")  
  
send_email(subject="Meeting Reminder", recipient="john@example.com", message="Don't forget our meeting at 10 AM.")  
  
# Output:  
# Sending email to john@example.com with subject 'Meeting Reminder' and message: Don't forget our meeting at 10 AM
```

Default Arguments

- A default argument has a **value already set in the function definition**.
- If you **don't pass a value**, the function uses the default.

Basic Usage

```
def greet(name="World"):  
    print(f"Hello, {name}!")
```

◆ Calling By Passing Argument

```
greet("Bob")  
  
# Output: Hello, Bob!
```

- "Bob" replaces the default value

◆ Without Passing Arguments

```
greet()  
  
# Output: Hello, World!
```

- When no value passed → **uses default: "World"**

Concept Practice Task

Greeting with a Default Name

```
def greet(name="Guest"):
    print(f"Hello, {name}!")

greet()

# Calling the function with a specific argument
greet("Alice")
```

```
# Output: # Hello, Guest!
          # Hello, Alice!
```

Creating a Message with Default Punctuation

```
def create_message(text, punctuation="."):
    print(f"{text}{punctuation}")

create_message("Hello")

create_message("Goodbye", "!")
```

```
# Output: # Hello.
          # Goodbye!
```

Sending a Reminder with Default Time

```
def send_reminder(task, time="09:00 AM"):
    print(f"Reminder: {task} at {time}.")

send_reminder("Team meeting")

send_reminder("Doctor's appointment", "03:00 PM")
```

```
# Output:
# Reminder: Team meeting at 09:00 AM.
# Reminder: Doctor's appointment at 03:00 PM.
```

Making a Pizza Order with Default Size

```
def order_pizza(size="Medium", toppings=None):
    if toppings is None:
        toppings = []
    print(f"Ordering a {size} pizza with toppings: {' '.join(toppings)}.")

order_pizza()

order_pizza(size="Large", toppings=["Pepperoni", "Mushrooms"])
```

```
# Output:
# Ordering a Medium pizza with toppings: .
# Ordering a Large pizza with toppings: Pepperoni, Mushrooms.
```

Return Statement

- What Is a return Statement?
- Using Return to Send Values Back
- Returning Multiple Values from a Function
- Return Statement vs. Print Statement

What is a Return Statement?

- The return statement is **used inside a function to send a value back to where the function was called.**
- It allows the function to **produce a result** instead of just printing something.

Basic Structure

```
def function_name(parameters):  
    # do something  
    return value
```



sends the value back

Explanation

- It **stops the function immediately.**
- It **sends a value back** to the part of the code that **called the function.**
- Any code after return won't be executed.

🎯 Why Use return?

- To get results from a **function** and use them elsewhere
- **To store the result in a variable**

Using Return to Send Values Back

Adding Two Numbers

```
def add_numbers(a, b):  
    total = a + b  
    return total
```

```
result = add_numbers(5, 3)  
print(result) # 8
```

Stores the return value

Returning a Square

```
def square(number):  
    result = number * number  
    return result
```

```
output = square(4)  
print(output) # 16
```



Note

- The return statement also denotes that the function has ended.
- Any code after return is not executed.
- Return statement can not be used outside the function.

Concept Practice Task

Sum of Two Numbers

```
def add_numbers(a, b):  
    return a + b  
  
result = add_numbers(5, 7)  
print(result)
```

```
# Output:  
# 12
```

Returning a String Message

```
def welcome_message():  
    return "Welcome to the Python course!"  
  
message = welcome_message()  
print(message)
```

```
# Output:  
# Welcome to the Python course!
```

Returning the Square of a Number

```
def square(num):  
    return num * num  
  
result = square(6)  
print(result)
```

```
# Output:  
# 36
```

Returning the First Item in a List

```
def first_item(items):  
    return items[0]  
  
result = first_item([10, 20, 30, 40])  
print(result)
```

```
# Output:  
# 10
```

Concept Practice Task

Factorial of a Number

```
def factorial(n):  
    result = 1  
    while n > 1:  
        result *= n  
        n -= 1  
    return result
```

```
result = factorial(5)  
print(result)
```

```
# Output: # 120
```

Maximum of Two Numbers

```
def maximum(a, b):  
    if a > b:  
        return a  
    else:  
        return b
```

```
result = maximum(10, 20)  
print(result)
```

```
# Output: # 20
```

Sum of a List of Numbers

```
def sum_of_list(numbers):  
    total = 0  
    for number in numbers:  
        total += number  
    return total
```

```
result = sum_of_list([5, 10, 15])  
print(result)
```

```
# Output: # 30
```

Length of a String

```
def string_length(s):  
    return len(s)
```

```
result = string_length("Hello, world!")  
print(result)
```

```
# Output:  
# 13
```

Returning Multiple Values from a Function

- In Python, a function can **return more than one value at the same time**.
- You do this by **separating the values with commas** in the return statement.
- Python will **automatically group them into a tuple**.

Basic Usage

```
def get_user_info():  
    name = "Alice"  
    age = 25  
    return name, age  
user_name, user_age = get_user_info()  
  
print("Name:", user_name)  
print("Age:", user_age)  
  
# Name: Alice  
# Age: 25
```

What's Happening:

- `get_user_info()` returns **two values**: "Alice" and 25
- Python treats this as **a tuple**: ("Alice", 25)
- Then we **use multiple variables** to store each **returned value separately**.

Concept Practice Task

Returning String Length and Uppercase

```
def string_info(s):  
    return len(s), s.upper()  
  
length, uppercase = string_info("hello")  
print(f"Length: {length}, Uppercase: {uppercase}")
```

```
# Output:  
# Length: 5, Uppercase: HELLO
```

Returning a Sum and Difference

```
def sum_and_difference(a, b):  
    return a + b, a - b  
  
result_sum, result_diff = sum_and_difference(10, 5)  
print(f"Sum: {result_sum}, Difference: {result_diff}")
```

```
# Output:  
# Sum: 15, Difference: 5
```

Returning Coordinates

```
def get_coordinates():  
    x = 10  
    y = 20  
    return x, y  
  
x_coord, y_coord = get_coordinates()  
print(f"Coordinates: ({x_coord}, {y_coord})")
```

```
# Output:  
# Coordinates: (10, 20)
```

Returning a Name and Age

```
def person_info():  
    name = "Alice"  
    age = 30  
    return name, age  
  
name, age = person_info()  
print(f"Name: {name}, Age: {age}")
```

```
# Output:  
# Name: Alice, Age: 30
```


Concept Practice Task

Multiple Properties of a Rectangle

```
def rectangle_properties(length, width):  
    area = length * width  
    perimeter = 2 * (length + width)  
    return area, perimeter  
  
area, perimeter = rectangle_properties(5, 3)  
print(f"Area: {area}, Perimeter: {perimeter}")
```

```
# Output:  
# Area: 15, Perimeter: 16
```

Returning Full Name and Initials

```
def full_name_and_initials(first_name, last_name):  
    full_name = f"{first_name} {last_name}"  
    initials = f"{first_name[0]}{last_name[0]}"  
    return full_name, initials  
  
full_name, initials = full_name_and_initials("John", "Doe")  
print(f"Full Name: {full_name}, Initials: {initials}")
```

```
# Output:  
# Full Name: John Doe, Initials: JD
```

Returning Status and Message

```
def check_value(value):  
    if value > 0:  
        return "Positive", "The value is greater than zero."  
    elif value < 0:  
        return "Negative", "The value is less than zero."  
    else:  
        return "Zero", "The value is zero."  
  
status, message = check_value(10)  
print(f"Status: {status}, Message: {message}")
```

```
# Output:  
# Status: Positive, Message: The value is greater than zero.
```

Return Statement vs. Print Statement

- Let's now look at how return and print are different.

Feature	print	return
• Purpose	→ Displays output on the screen	→ Sends a value back from a function
• Where to Use	→ Can be used anywhere in the code	→ Only used inside a function
• Function Behavior	→ Does not stop the function	→ Ends the function immediately
• Value Handling	→ Just shows the value, doesn't store it	→ Can be stored in a variable
• Output Location	→ Output appears in the console	→ Output goes back to the caller
• Multiple Outputs	→ Can print many values repeatedly	→ Can return multiple values once

Variable Scopes

- What is Variable Scope ?
- Global Scope
- Local Scope
- Using the global Keyword

What is Variable Scope ?

- **Scope** means the **part of the code where a variable exists** and can be **accessed**.
- Understanding scope is **important because** it tells you:
 - Where a **variable is available**
 - What **happens to variables inside vs. outside functions**
 - How to **avoid naming issues or errors**

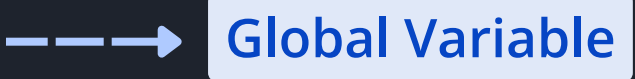
Ways of Variable Scopes

- **Global Scope**
 - Variables declared **outside any function**
 - They can be **accessed anywhere** in the code
- **Local Scope**
 - Variables declared **inside a function**
 - **Accessible** only **within that function**
- **Using global Keyword**
 - Used inside a function to **modify a global variable**

Global Scope

- A global variable is **created outside any function**.
- It can be **accessed from anywhere — inside or outside a function**.

```
x = 10
```



```
def my_function():  
    print("Inside function, x =", x)
```



access inside function

```
my_function()  
print("Outside function, x =", x)
```



access Outside function

```
# Inside function, x = 10  
# Outside function, x = 10
```

Concept Practice Task

Reading a Global Variable

```
message = "Hello, World!"

def print_message():
    print(message)

print_message()
```

```
# Output:
# Hello, World!
```

Reading Global Variable with Conditional Logic

```
threshold = 10

def check_value(value):
    if value > threshold:
        print("Value is above the threshold.")
    else:
        print("Value is below or equal to the threshold.")

check_value(15)
check_value(5)
```

```
# Output:
# Value is above the threshold.
# Value is below or equal to the threshold.
```

Using Global Variable for Configuration

```
config = "development"

def display_config():
    print(f"Current configuration: {config}")

display_config()
```

```
# Output:
# Current configuration: development
```

Calculating with Global Variable

```
multiplier = 2

def calculate_product(value):
    product = value * multiplier
    print(f"Product: {product}")

calculate_product(5)
```

```
# Output:
# Product: 10
```

Concept Practice Task

Calculating with Global Variable

```
multiplier = 2

def calculate_product(value):
    product = value * multiplier
    print(f"Product: {product}")

calculate_product(5)
```

```
# Output:
# Product: 10
```

Reading Global Variable with Conditional Logic

```
threshold = 10

def check_value(value):
    if value > threshold:
        print("Value is above the threshold.")
    else:
        print("Value is below or equal to the threshold.")

check_value(15)
check_value(5)
```

```
# Output:
# Value is above the threshold.
# Value is below or equal to the threshold.
```

Local Scope

- A local variable is **created inside a function** and only **exists inside that function**.
- Once the **function is finished**, the **variable discarded** & can't access it outside of that function.

```
def my_function():  
    x = 5  
    print("Inside function, x =", x)
```

Local variable

```
my_function()  
# Output: Inside function, x = 5
```

```
print(x)
```

What if we try to access it outside the code?

```
ERROR!  
Traceback (most recent call last):  
  File "<main.py>", line 8, in <module>  
NameError: name 'x' is not defined
```

- Because **x was created inside the function**, it only lives there.
- Once the **function ends**, **x is no longer available**.

Concept Practice Task

GamutX

Basic Local Variable Declaration

```
def greet():  
    greeting = "Hello, World!"  
    print(greeting)
```

```
greet()
```

```
# Output:  
# Hello, World!
```

String Manipulation

```
def convert_to_uppercase(text):  
    upper_text = text.upper()  
    print("Uppercase text:", upper_text)
```

```
convert_to_uppercase("hello, world!")
```

```
# Output:  
# Uppercase text: HELLO, WORLD!
```

Performing Arithmetic Operations

```
def add_numbers(a, b):  
    sum_result = a + b  
    print("The sum is:", sum_result)
```

```
add_numbers(5, 7)
```

```
# Output:  
# The sum is: 12
```

Summing Values in a List

```
def sum_list(numbers):  
    total = 0 # Local variable  
    for number in numbers:  
        total += number  
    print("Total sum:", total)
```

```
sum_list([1, 2, 3, 4, 5])
```

```
# Output:  
# Total sum: 15
```

Evaluating Even or Odd

```
def check_even_odd(number):  
    if number % 2 == 0:  
        result = "Even"  
    else:  
        result = "Odd"  
    print(f"The number {number} is {result}.")
```

```
check_even_odd(10)  
check_even_odd(3)
```

```
# Output:  
# The number 10 is Even.  
# The number 3 is Odd.
```


Using the global Keyword

- By default, any **variable we create inside a function is local**.
- to create a **global variable inside a function we use global keyword**.
- **Useful for tracking state across multiple function calls**

Global keyword

- makes variable x **accessible outside function too**

Global Variable

```
x = 10
print("Before modification, x =", x)

def my_function():
    global x
    x = 20
    print("Inside function, x =", x)

my_function()
print("After modification, x =", x)

# Before modification, x = 10
# Inside function, x = 20
# After modification, x = 20
```

Global , Local & Global Keyword Summary

Local variable

- Where it Defined
Inside Function
- Accessible
Only inside that function

Global keyword

- Where it Defined
Inside Function
- Accessible
Lets you modify a global variable variable inside a function

```
x = 5

def demo():
    y = 10
    global z
    z = 15

    print("Inside function:")
    print("x =", x)
    print("y =", y)
    print("z =", z)

demo()
```

Global Variable

- Where it Defined
Outside function
- Accessible
Anywhere in the code

Concept Practice Task

Modifying a Global Variable

```
count = 0

def increment():
    global count
    count += 1

increment()
increment()
print(count)
```

Output: # 2

Using Global Variable in Multiple Function

```
total = 100

def add_to_total(amount):
    global total
    total += amount

def subtract_from_total(amount):
    global total
    total -= amount

add_to_total(50)
subtract_from_total(30)
print(total)
```

Output: # 120

Global Variable in Conditional Statement

```
temperature = 25

def check_temperature():
    global temperature
    if temperature > 30:
        print("It's hot!")
    else:
        print("It's pleasant.")

check_temperature()
```

Output: # It's pleasant.

Concept Practice Task

GamutX

Global Variable for Game Score

```
score = 0

def increase_score(points):
    global score
    score += points

def reset_score():
    global score
    score = 0

increase_score(10)
print("Score after increase:", score)

reset_score()
print("Score after reset:", score)
```

```
# Output:
# Score after increase: 10
# Score after reset: 0
```

Using Global Variables to Track State

```
is_running = False

def start_process():
    global is_running
    is_running = True
    print("Process started.")

def stop_process():
    global is_running
    is_running = False
    print("Process stopped.")

start_process()
print("Is running:", is_running)

stop_process()
print("Is running:", is_running)
```

```
# Output:
# Process started.
# Is running: True
# Process stopped.
# Is running: False
```

Docstring

- What is a Docstring?
- How to Write a Docstring
- Accessing Docstrings
- Using `__doc__` Attribute
- Using `help()` Function
- Comments vs Docstrings

What is a Docstring?

A **docstring** is a **special kind of comment written inside a function, class, or module**

Think of it like a short instruction manual for code

- What the function does ?
- What inputs it takes ?
- What it returns (if anything) ?

🔍 Real-Life Analogie

1. User Manual for a Gadget

- **Function** = The device doing the job
- **Docstring** = The user guide telling you what the device does

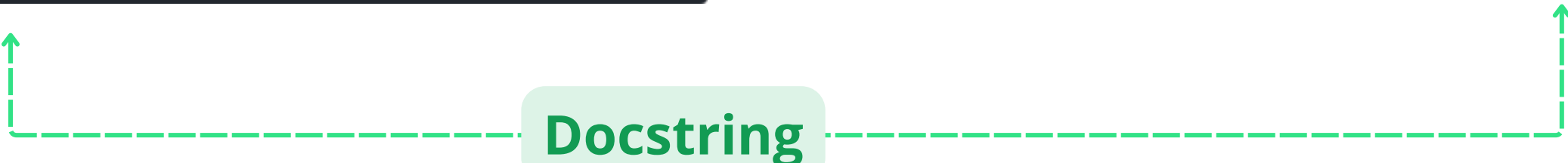
```
def take_photo():  
    """This function captures a photo using the phone's camera."""
```

2. GPS Directions

- **Function** = The process that gives directions
- **Docstring** = A summary of what the trip will cover

```
def get_directions(start, end):  
    """This function provides directions from the start location to the end location."""
```

Docstring



How to Write a Docstring

We write a **docstring** just below the function definition, **using triple quotes** (""" """).

```
def greet(name):  
    """  
    This function greets the person whose name is passed as a parameter.  
  
    Parameters:  
    name (str): The name of the person to greet.  
  
    Returns:  
    None  
    """  
    print(f"Hello, {name}!")
```

Brief description of what the function does.

- **Name** – The variable's name (e.g., name)
- **Type** – The kind of value it expects (e.g., str, int)
- **Description** – What the parameter is used for

- we write the **type of value the function gives back**
- A **short description of what is returned**.

🔍 What this docstring tells us:

- What the **function does**
- What **parameter it takes**, including the **type (str)**
- What **it returns** (None in this case, because it only prints)

Accessing Docstrings

Once we've added a docstring to a function, you can access it in two main ways:

- Using the `__doc__` attribute
- Using the built-in `help()` function

Both methods let you see the **explanation written in the docstring**,

Using `__doc__` Attribute

We can **access** a function's docstring by using the special attribute `__doc__`.

```
def greet(name):  
    """  
    This function greets the person whose name is passed as a parameter.  
  
    Parameters:  
    name (str): The name of the person to greet.  
  
    Returns:  
    None  
    """  
    print(f"Hello, {name}!")  
  
print(greet.__doc__)
```

Output

```
# Output  
    This function greets the person whose name is passed as a parameter.  
  
    Parameters:  
    name (str): The name of the person to greet.  
  
    Returns:  
    None
```

When to Use:

- When we **want to quickly check the function's purpose** within your code
- Good for **debugging** or **reviewing code** from others

Concept Practice Task

GamutX

Simple Function with Docstring

```
def greet():  
    """Display a greeting message."""  
    print("Hello, World!")  
  
print(greet.__doc__)
```

```
# Output:  
# Display a greeting message.
```

Function with Parameters & Docstring

```
def add(a, b):  
    """Return the sum of two numbers."""  
    return a + b  
  
print(add.__doc__)
```

```
# Output:  
# Return the sum of two numbers.
```

Function Describing Its Purpose

```
def multiply(x, y):  
    """Multiply x and y and return the result."""  
    return x * y  
  
print(multiply.__doc__)
```

```
# Output:  
# Multiply x and y and return the result.
```

Function with Detailed Docstring

```
def maximum(a, b, c):  
    """  
    Return the maximum of three numbers.  
  
    Parameters:  
    a (int or float): First number  
    b (int or float): Second number  
    c (int or float): Third number  
    """  
    return max(a, b, c)  
  
print(maximum.__doc__)
```

```
# Output:  
# Return the maximum of three numbers.  
#  
# Parameters:  
# a (int or float): First number  
# b (int or float): Second number  
# c (int or float): Third number
```

Using help() Function

- `help()` is a built-in function in **Python** that shows **you documentation** for
 - **Functions**
 - **Classes**
 - **Modules**
 - **Other Python objects**

Let's say we have a simple function:

```
def greet():  
    """Display a greeting message."""  
    print("Hello, World!")
```

```
help(greet)
```

```
# Help on function greet in module __main__:  
  
# greet()  
#     Display a greeting message.
```

🎯 Purpose of help()

- To view the **docstring** of a function or class
- To understand how to **use a function**
- **Helpful** when **working with new code** or **built-in Python libraries**

Concept Practice Task

Function to Add Two Numbers

```
def add(a, b):  
    """Return the sum of two numbers."""  
    return a + b  
  
help(add)
```

```
#Output:  
Help on function add in module __main__:  
  
add(a, b)  
    Return the sum of two numbers.
```

Function to Check if a Number is Even

```
def is_even(n):  
    """Return True if the number is even, otherwise False."""  
    return n % 2 == 0  
  
help(is_even)
```

```
#Output:  
Help on function is_even in module __main__:  
  
is_even(n)  
    Return True if the number is even, otherwise False.
```

Function to Convert Celsius to Fahrenheit

```
def celsius_to_fahrenheit(celsius):  
    """Convert Celsius temperature to Fahrenheit."""  
    return (celsius * 9/5) + 32  
  
help(celsius_to_fahrenheit)
```

```
# Output:  
# Help on function celsius_to_fahrenheit in module __main__:  
#  
# celsius_to_fahrenheit(celsius)  
#     Convert Celsius temperature to Fahrenheit.
```

Function to Find the Minimum of Two Numbers

```
def minimum(a, b):  
    """Return the minimum of two numbers."""  
    return a if a < b else b  
  
help(minimum)
```

```
# Output:  
# Help on function minimum in module __main__:  
#  
# minimum(a, b)  
#     Return the minimum of two numbers.
```

Comments vs Docstrings

- Both serve different purposes and are used in different ways.

Feature	Comments	Docstrings
• Purpose	→ Explain code logic for developers	→ Document functions, classes, or modules
• Syntax	→ Starts with #	→ Enclosed in triple quotes """ ... """
• Location	→ Anywhere in the code	→ At the beginning of functions, classes, or modules
• Execution	→ Ignored by the Python interpreter	→ Can be accessed at runtime using <code>.__doc__</code>
• Use Case	→ Quick notes or reminders in the code	→ Detailed documentation for users and developers
• Access	→ Not accessible during program execution	→ Can be accessed using <code>__doc__</code> or <code>help()</code>

Note

- **Comments:** For developer understanding.
- **Docstrings:** For documentation and runtime access.

The Range Function

- What is range() Function ?
- Basic Syntax
- Understanding range(stop)
- Understanding - range(start, stop)
- Understanding range(start, stop, step)

What is range() Function ?

- The range() function is used to **generate a sequence of numbers**.
- It's **most commonly used in for loops** to repeat something a certain number of times.

Different Ways to Use range()

- **range(stop):**
 - Generates a sequence **from 0 up to but not including stop**.
- **range(start, stop):**
 - Generates a sequence **from start up to but not including stop**.
- **range(start, stop, step):**
 - Generates a sequence **from start up to but not including stop**, with a **specified step value**.

Basic Syntax

range(stop)

```
range(stop)
```

- Starts from **0 by default**
- Goes up to, but does **not include, stop**

range(start, stop)

```
range(start, stop)
```

- Starts from the **given start value**
- Goes up to **but not including stop**

range(start, stop, step)

```
range(start, stop, step)
```

- **start** → The number where the **sequence begins**.
- **stop** → The number where the **sequence ends (but not included)**.
- **step** → **How much to add** each time to **get the next number**.

Understanding range(stop)

- Starts from 0 and goes up to but not including stop.

```
for i in range(5):  
    print(i)
```

↓
Stop

Output:

```
# 0  
# 1  
# 2  
# 3  
# 4
```

Iteration Breakdown

Iteration 1:

- $i = 0$
- 🖱️ Output: 0

Iteration 2:

- $i = 0$
- 🖱️ Output: 0

Iteration 3:

- $i = 2$
- 🖱️ Output: 2

Iteration 4:

- $i = 3$
- 🖱️ Output: 3

Iteration 5:

- $i = 4$
- 🖱️ Output: 4

Useful when we want to run a loop N times, starting from 0

Concept Practice Task

Sum of First n Numbers

```
total_sum = 0
for number in range(10):
    total_sum += number

print("Sum of numbers from 0 to 9:", total_sum)
```

```
# Output:
# Sum of numbers from 0 to 9: 45
```

Iterating Over a List with Index

```
fruits = ["apple", "banana", "cherry"]
for i in range(len(fruits)):
    print(f"Index {i}: {fruits[i]}")
```

```
# Output:
# Index 0: apple
# Index 1: banana
# Index 2: cherry
```

Finding Even Numbers

```
print("Even numbers from 0 to 9:")
for num in range(10):
    if num % 2 == 0:
        print(num)
```

```
# Output:
# Even numbers from 0 to 9:
# 0
# 2
# 4
# 6
# 8
```

Printing Odd Numbers

```
print("Odd numbers from 0 to 9:")
for num in range(10):
    if num % 2 != 0:
        print(num)
```

```
# Output:
# Odd numbers from 0 to 9:
# 1
# 3
# 5
# 7
# 9
```

Understanding - range(start, stop)

- Starts from start and goes up to but not including stop.

```
for i in range(2, 7):  
    print(i)
```

↓ Start
↘ Stop

Output:

```
# 2  
# 3  
# 4  
# 5  
# 6
```

Iteration Breakdown

Iteration 1:

- $i = 2$
- 🖱️ Output: 2

Iteration 4:

- $i = 5$
- 🖱️ Output: 5

Iteration 2:

- $i = 3$
- 🖱️ Output: 3

Iteration 5:

- $i = 6$
- 🖱️ Output: 6

Iteration 3:

- $i = 4$
- 🖱️ Output: 4

Concept Practice Task

Filling a List with Values from a Range

```
squares = []
for i in range(1, 6): # Squares from 1 to 5
    squares.append(i ** 2)

print("Squares of numbers from 1 to 5:", squares)
```

```
# Output:
# Squares of numbers from 1 to 5: [1, 4, 9, 16, 25]
```

Simple Multiplication Table

```
print("Multiplication table for 3:")

for i in range(1, 11):
    print(f"3 x {i} = {3 * i}")
```

```
# Multiplication table for 3:
# 3 x 1 = 3
# 3 x 2 = 6
# 3 x 3 = 9
# 3 x 4 = 12
# 3 x 5 = 15
# 3 x 6 = 18
# 3 x 7 = 21
# 3 x 8 = 24
# 3 x 9 = 27
# 3 x 10 = 30
```

Checking Perfect Squares

```
print("Perfect squares between 1 and 100:")

for i in range(1, 11):
    perfect_square = i ** 2
    print(perfect_square)
```

```
# Perfect squares between 1 and 100:
# 1
# 4
# 9
# 16
# 25
# 36
# 49
# 64
# 81
# 100
```

Generating Fibonacci Sequence

```
fib = [0, 1]
n = 10

for i in range(2, n):
    next_fib = fib[-1] + fib[-2]
    fib.append(next_fib)

print("Fibonacci sequence:", fib)
```

```
# Output:
# Fibonacci sequence: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Understanding range(start, stop, step)

- Starts from start, ends before stop, and increases by step.

```
for i in range(1, 11, 2):  
    print(i)
```

Output:

1
3
5
7
9

Start
Stop
Steps

Iteration Breakdown

Iteration 1:

- $i = 1$
- 👉 Output: 1

Iteration 2:

- $i = 3$
- 👉 Output: 3

Iteration 3:

- $i = 5$
- 👉 Output: 5

Iteration 4:

- $i = 7$
- 👉 Output: 7

Iteration 5:

- $i = 9$
- 👉 Output: 9

This is useful when we want to skip values or use even/odd filtering

Concept Practice Task

Printing a Sequence of Numbers

```
print("Sequence of numbers from 0 to 20 with a step of 5:")
for num in range(0, 21, 5):
    print(num)
```

```
# Output:
# Sequence of numbers from 0 to 20 with a step of 5:
# 0
# 5
# 10
# 15
# 20
```

Iteration with a Negative Step

```
for i in range(10, 0, -1):
    print(i)
```

```
#Output:
10
9
8
7
6
5
4
3
2
1
```

Skipping Elements in a List

```
numbers = [10, 20, 30, 40, 50, 60, 70]
print("Every second number from the list:")
for i in range(0, len(numbers), 2):
    print(numbers[i])
```

```
# Output:
# Every second number from the list:
# 10
# 30
# 50
# 70
```

Concept Practice Task

GamutX

Printing Odd Number

```
print("Odd numbers from 1 to 19:")
for num in range(1, 20, 2):
    print(num)
```

```
# Output:
# Odd numbers from 1 to 19:
# 1
# 3
# 5
# 7
# 9
# 11
# 13
# 15
# 17
# 19
```

Generating a Reverse Sequence of Cubes

```
print("Cubes of numbers from 5 to 1:")
for num in range(5, 0, -1):
    print(f"{num}^3 = {num ** 3}")
```

```
# Output:
# Cubes of numbers from 5 to 1:
# 5^3 = 125
# 4^3 = 64
# 3^3 = 27
# 2^3 = 8
# 1^3 = 1
```


Enumeration in Python

- What is Enumeration?
- Problem Scenario
- Syntax of enumerate()
- Looping Over an Enumerate Object
- Enumerating a String
- Enumerating a List
- Iteration with start Argument
- Enumerating a List of Lists
- Enumerating a Dictionary
- Difference - range() + len() & enumerate()
- Why Use enumerate()?

What is Enumeration?

- When we're working with lists, tuples, or any collection of items — we often want **two things**:
 - The **value** (like the artist's name, the shot ID, or file path)
 - The **position** of that value (its index)
- This process of **getting both at the same time is called enumeration.**
- Python gives us a built-in function called **enumerate()** to handle this easily.

Problem Scenario

Let's say you're looping over a list of review notes, and you want to number them: You could do this:

```
notes = ['Fix lighting on shot 03', 'Add motion blur', 'Color grade pass needed']

for i in range(len(notes)):
    print(i, notes[i])
```

Output:
0 Fix lighting on shot 03
1 Add motion blur
2 Color grade pass needed

Manual index-based looping
Prints index and item separately
Access item using index

But this is clunky. There's a better way.

Use enumerate()

```
for index, note in enumerate(notes):
    print(index, note)
```

Unpacks position & item
Automatically provides index and value

✓ Now you get both the position and the content in one clean line.

Syntax of enumerate()

Syntax →

```
enumerate(iterable, start=0)
```

Built-in function

Any sequence

- list, string, tuple, dictionary, etc.

Starting value for the index (*optional*)

- By default its 0

📌 Important to remember:

- enumerate() **does not return a list.**
- It **returns** an **enumerate object.**

Looping Over an Enumerate Object

1 print the enumerate object itself.

```
items = ['A', 'B', 'C']  
result = enumerate(items)  
  
print(result)
```

Creates enumerate object

Output: <enumerate object at 0x0000021F8A4B>

3 Unpacking Object

```
for index, item in enumerate(items):  
    print(index, item)
```

Output:

```
# 0 A  
# 1 B  
# 2 C
```

2 Looping Over an Enumerate Object

```
for value in enumerate(items):  
    print(value)
```

Tuple containing index and item

Output

```
(0, 'A')  
(1, 'B')  
(2, 'C')
```

Each loop iteration gives: → (index, value)

Enumerating a String

Strings are sequences of characters, so they can also be enumerated.

```
text = "VFX"
```

```
for index, char in enumerate(text):  
    print(index, char)
```

value

position of each char

```
"""
```

```
Output:
```

```
0 V
```

```
1 F
```

```
2 X
```

```
"""
```

📌 Use case:

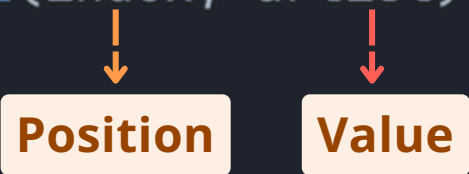
- Character-level processing
- Text tools
- Validation or parsing tasks

Enumerating a List

This is the most common and practical use case.

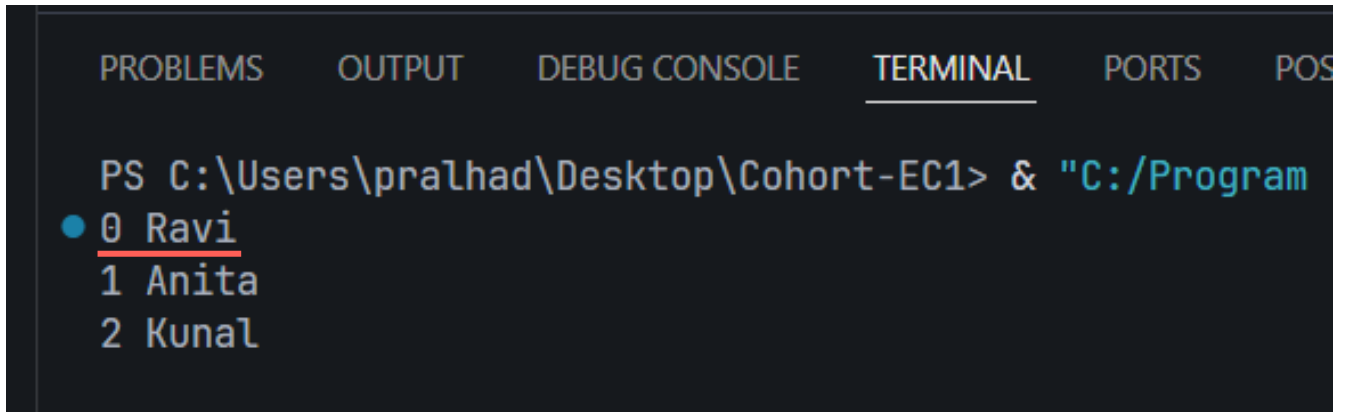
```
artists = ['Ravi', 'Anita', 'Kunal']  
  
for index, artist in enumerate(artists):  
    print(index, artist)
```

`print()` **Position** **Value**



Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POS  
  
PS C:\Users\pralhad\Desktop\Cohort-EC1> & "C:/Program  
● 0 Ravi  
1 Anita  
2 Kunal
```



Iteration with start Argument

By default, `enumerate()` starts counting from 0.

Example with start

```
tasks = ['Modeling', 'Texturing', 'Lighting']  
for index, task in enumerate(tasks, start=1):  
    print(index, task)
```

Changes starting index value

'''

Output:

```
1 Modeling  
2 Texturing  
3 Lighting
```

'''

📌 Key Points

- **start** does **not** change the data
- It only changes **how the index is counted**
- We can start from **any number**, not just 1

Enumerating a List of Lists

```
shots = [  
    ['shot01', 'Approved'],  
    ['shot02', 'Pending'],  
    ['shot03', 'Fix Required']  
]
```

```
for index, shot in enumerate(shots, start=1):  
    print(index, shot)
```

Output:

```
1 ['shot01', 'Approved']  
2 ['shot02', 'Pending']  
3 ['shot03', 'Fix Required']
```

Index

inner List

Now let's **unpack** it properly:

index position

Unpacks inner list values

```
for index, (shot_name, status) in enumerate(shots, start=1):  
    print(index, shot_name, status)
```

Inner List Elements

Output:

```
1 shot01 Approved  
2 shot02 Pending  
3 shot03 Fix Required
```

Inner List

Enumerating a Dictionary

When enumerating a dictionary, Python loops over **keys by default**.

```
shots = {  
    'shot01': 'Approved',  
    'shot02': 'Pending',  
    'shot03': 'Fix Required'  
}
```

```
for index, key in enumerate(shots):  
    print(index, key)
```

Output:

```
0 shot01  
1 shot02  
2 shot03
```

Keys

Now let's **unpack it properly**:

Tuple from dictionary item

Starts index counting
from one

```
for index, (key, value) in enumerate(shots.items(), start=1):  
    print(index, key, value)
```

Returns key-value pairs

- Each item looks like: (key, value)

Output:

```
1 shot01 Approved  
2 shot02 Pending  
3 shot03 Fix Required
```

Difference - range() + len() & enumerate()

When we want to loop through a list and get both the index and the **value**, there are two ways:

1. Using range() with len()

```
tasks = ['Compositing', 'Lighting', 'Matchmove']  
  
for i in range(len(tasks)): --> Manual index-based looping  
    print(i, tasks[i])
```

- We use range() to generate numbers
- Then use each number as an index to access the list
- This method is manual — two steps: get the index, then get the item

2. Using enumerate()

```
tasks = ['Compositing', 'Lighting', 'Matchmove']  
  
for i, task in enumerate(tasks): -->  
    print(i, task)  
    Provides index and item together
```

- We get both index and item directly
- No need to use len() or access tasks[i] manually
- This method is cleaner and easier to understand

Key Difference

Point

`range(len(...))``enumerate()`

- | | | |
|-----------------------------|------------------------------|-----------------------------------|
| • Code style | → Manual indexing | → Direct access to index and item |
| • Readability | → Slightly more complex | → Cleaner and easier to read |
| • Error chances | → Higher (indexing mistakes) | → Lower |
| • Custom start index | → Not supported | → Supported (start=) |

Which one to use?

- Use `enumerate()` **when both the index and the value are needed.**
- It's more readable and avoids manual mistakes.

Why Use enumerate()?

When looping over a **list**, we often need both:

- The **item** itself
- The **position** (index) of that item

To Solve This we can

range() with len((items))

```
for i in range(len(tasks)):
    print(i, tasks[i])
```

enumerate()

Better way

```
for i, task in enumerate(tasks):
    print(i, task)
```

WHY ?

enumerate() is preferred because:

- ✓ Gives index and item in one step
- ✓ Makes code shorter and cleaner
- ✓ No need to use list[i] manually
- ✓ Reduces indexing mistakes
- ✓ Allows starting index change using start=1

Thank You

